

Fortran vs JS/DCP — fidelity & performance

seed 42 · 10 realization(s)

SHOW ☒ ☐ fortran-local ☒ ☐ js-local

JS-local total job time, vs fortran-local
59% slower
15.4s → 24.3s

JS-local per-realization time, vs fortran-local
129% slower
2.2s → 5.0s per realization (median)

Run parameters				
Reproducibility metadata for each leg, recorded at the time it ran.				
LEG	SEED	BOOTSTRAPS	CPUS USED	CLOUD DENSITY
● fortran-local	42	10	4 / 8 cores	100
● js-local	42	10	4 / 8 cores	100

Performance					
Total wall time and per-realization breakdown, by run. Per-realization/breakdown figures are the median across realizations, excluding any realization whose evalCount (or, for legs that don't report it, totalMs) sits more than 5× MAD from the median -- evalCount is preferred where available since it isolates genuine algorithmic difficulty (the optimizer needing far more evaluations) from a realization simply landing on a slow/contented worker, which is not the same thing and shouldn't be treated as an outlier.					
LEG	TOTAL WALL	PER-REALIZATION	RESAMPLE	SOFIA	FIT
● fortran-local	15.4s	2.2s	64ms	32ms	2052ms
● js-local	24.3s	5.0s	71ms	56ms	4679ms

Overall fit results								
Median ± 1.4825×MAD across all realizations, same statistic Bootstrap_Error_Analysis.py itself uses. Uncertainty to 2 significant figures; median rounded to match.								
LEG	N	PA (°)	INC (°)	VSYS	Xc	Yc	R _{h,i} (")	V _{h,i}
● fortran-local	10	88 ± 20	51 ± 11	913 ± 10	23.0 ± 1.1	21.88 ± 0.84	46.0 ± 3.4	60 ± 22
● js-local	10	88 ± 20	51 ± 11	913 ± 10	23.0 ± 1.1	21.88 ± 0.84	46.0 ± 3.4	60 ± 22

Individual fit results								
Every realization, grouped by realization index.								
▼ Show all 10 realization(s)								
LEG	#	PA (°)	INC (°)	VSYS	Xc	Yc	R _{h,i} (")	V _{h,i}
● fortran-local	0	178.4505	16.8977	919.9953	22.6194	21.9613	35.5495	79.1988
● js-local	0	178.4505	16.8977	919.9953	22.6194	21.9613	35.5495	79.1988
● fortran-local	1	70.1503	39.2776	919.4954	21.5617	21.6635	48.1420	53.9414
● js-local	1	70.1503	39.2776	919.4954	21.5617	21.6635	48.1420	53.9414
● fortran-local	2	92.7507	57.6424	914.0867	22.5268	22.5292	43.5980	66.8660
● js-local	2	92.7507	57.6424	914.0867	22.5268	22.5292	43.5980	66.8660
● fortran-local	3	20.8352	42.6861	898.1975	24.5585	20.9128	44.3726	65.4480
● js-local	3	20.8352	42.6861	898.1975	24.5585	20.9128	44.3726	65.4480

LEG	#	PA (°)	INC (°)	VSYS	Xc	Yc	R _{h,i} (")	V _{h,i}
● fortran-local	4	86.7944	62.5090	915.3948	22.1408	22.9983	45.9694	17.3784
● js-local	4	86.7944	62.5090	915.3948	22.1408	22.9983	45.9694	17.3784
● fortran-local	5	42.6658	51.4434	899.0045	23.9066	21.3964	46.0349	49.7302
● js-local	5	42.6658	51.4434	899.0045	23.9066	21.3964	46.0349	49.7302
● fortran-local	6	97.7699	51.1725	911.4393	23.4599	22.0070	49.0510	79.3965
● js-local	6	97.7699	51.1725	911.4393	23.4599	22.0070	49.0510	79.3965
● fortran-local	7	89.3699	51.2654	915.5256	23.7999	22.9856	46.7892	19.2286
● js-local	7	89.3699	51.2654	915.5256	23.7999	22.9856	46.7892	19.2286
● fortran-local	8	78.8309	40.5299	885.3690	23.7045	20.8295	49.3130	94.8941
● js-local	8	78.8309	40.5299	885.3690	23.7045	20.8295	49.3130	94.8941
● fortran-local	9	291.3878	56.0989	881.3136	22.3727	21.8062	29.2265	53.7428
● js-local	9	291.3878	56.0989	881.3136	22.3727	21.8062	29.2265	53.7428

Radial profiles

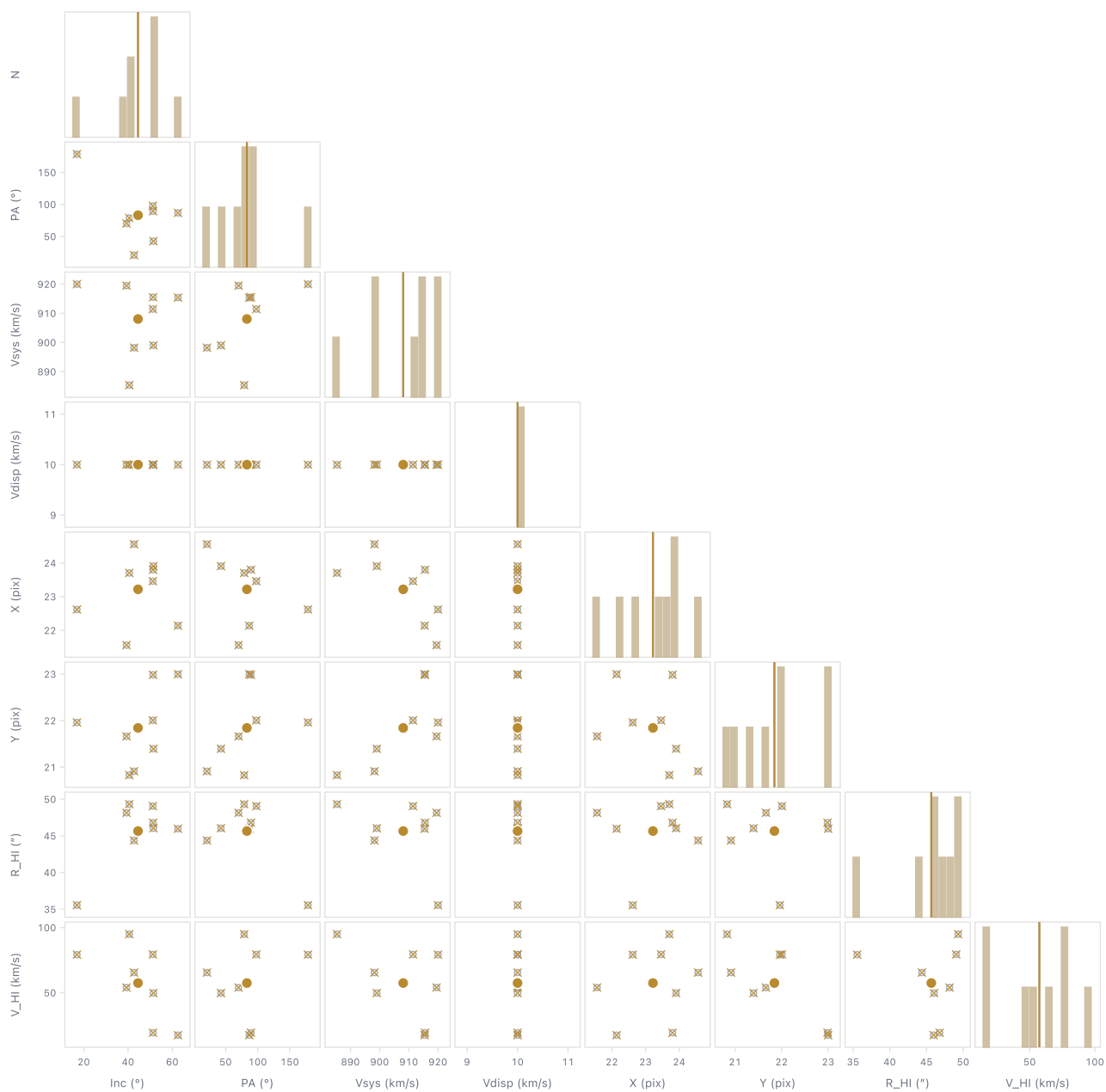
V_{rot} and Σ per ring (arcsec radius) — median ± 1.4825×MAD across all realizations of each run.

LEG	R (")			V _{ROT}		
● fortran-local	7.53, 22.58, 37.63, 52.68	33.2±34.2, 53.5±40.9, 57.7±40.5, 34.0±30.5	4.57±1.08, 1.28±0.47, 1.59±1.16, 0.			
● js-local	7.53, 22.58, 37.63, 52.68	33.2±34.2, 53.5±40.9, 57.7±40.5, 34.0±30.5	4.57±1.08, 1.28±0.47, 1.59±1.16, 0.			

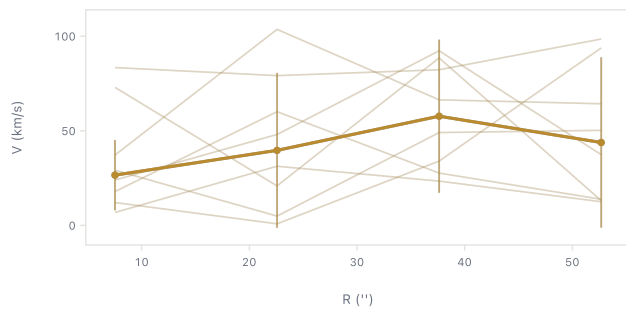
Geometry & profiles

Bootstrap realization distributions, overlaid by leg (use the Show filter above to isolate one). Small dots are individual realizations; the larger outlined dot/line per leg is that leg's own **mean across its realizations** (not a separate single best-fit — run_all_three only dispatches bootstrap realizations, no distinct reference fit is captured).

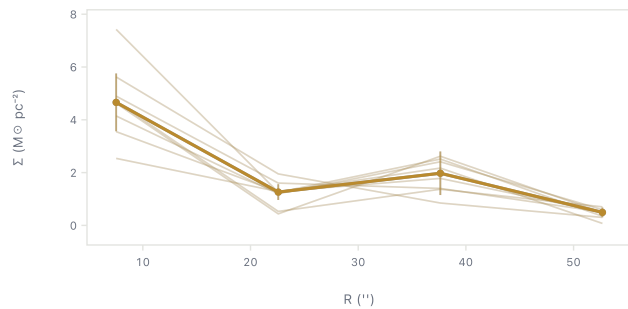
4 realization(s) excluded from the plots below as outliers (non-finite, or a plotted value more than 5× MAD from that leg's own median).



ROTATION CURVE



SURFACE DENSITY



Numerical agreement

fortran-local vs js-local — median of all bootstrap realizations per parameter for each leg (10 vs 10 realization(s)), same statistic as Overall fit results, and the absolute/percent difference between those two medians.

FIELD	FORTTRAN-LOCAL MEDIAN	JS-LOCAL MEDIAN	DIFF	% DIFF
PA (°)	88.082	88.082	0	0%
Inc (°)	51.219	51.219	0	0%
VSys	912.763	912.763	0	0%
Xc	23.040	23.040	0	0%

FIELD	FORTTRAN-LOCAL MEDIAN	JS-LOCAL MEDIAN	DIFF	% DIFF
Yc	21.884	21.884	0	0%
R _{hi} (")	46.002	46.002	1.625e-7	3.53e-7%
V _{hi}	59.695	59.695	7.177e-7	0.00000120%

On the residual ~0.1–1% fortran-vs-JS/DCP gap (PA, Inc, R_{hi}, V_{hi}, etc., after the PA-conversion and noise-corner-sampling fixes): traced with matched-seed, per-realization tracing on both sides. The resampled cube, SoFiA catalogue (`ellMaj` / `ellMin` / `kin_pa`), geometry estimate (`paEst` / `incEst`), and noise RMS all matched to 7–9 significant figures between languages for the same realization — i.e. everything the optimizer receives as input is effectively identical. The remaining gap therefore accumulates inside the ~400+ iteration Nelder-Mead optimization itself: ordinary floating-point non-associativity between a compiled-Fortran and a JIT-compiled JS/WASM pipeline, amplified on the way out by the well-known inclination/PA degeneracy in tilted-ring fitting (which is also why RHI/VHI, which inherit that degeneracy, show the largest residual % while well-conditioned parameters like Vsys and Y stay tight). Not a discovered bug — expected cross-language floating-point noise, pending a full per-evaluation trace diff if a tighter bound is ever needed.